Assoc.Prof. Kultida Rojviboonchai
Karnkitti Kittikamron (TA)

Edge–Cloud Computing

Use Case: Video and Image Processing Services

Objectives

1. To show the use case and the design of edge, cloud, and edge–cloud computing system.
2. To compare system performance in terms of latency, bandwidth usage, and computational load.
3. To point out the advantages of an edge–cloud architecture for real-time applications.

Overview

Video and Image processing play a crucial role in modern applications such as smart surveillance, healthcare, retail analytics, and autonomous driving. However, these applications require high computational resources and often generate large volumes of data, leading to increased server costs and network latency when processed exclusively in the *cloud*. On the other hand, *edge computing* reduces latency and saves bandwidth by processing data locally, but it is limited by lower computational capacity and difficulty scaling for large workloads.

An edge–cloud architecture offers a balanced approach between **edge nodes** which handle time-sensitive, lightweight processing close to the data source, reducing latency and bandwidth usage and **cloud servers** which perform more resource-intensive tasks such as model training, large-scale analytics, and long-term storage. This combination enables real-time decision-making at the edge, while leveraging the scalability and computational strength of cloud.

The use of edge–cloud computing in image processing will be demonstrated through the following activity. This activity highlights how splitting workloads between edge and cloud reduces latency, optimizes bandwidth, and improves efficiency.

Prerequisite

1. Personal Computer

1.1. Install Docker https://www.docker.com/products/docker-desktop/.

1.2. Download activity code repository using git.

```
git clone https://github.com/paychorn/sds-edge-computing.git
cd sds-edge-computing
```

1.3. Setup python environment

1.3.1. Create a virtual environment.

For Linux/macOS	`python3 -m venv sds`
For Windows	`python -m venv sds`

1.3.2. Activate the virtual environment.

For Linux/macOS	`source ./sds/bin/activate`
For Windows (CommandPrompt)	`.\sds\Scripts\activate`
For Windows (Powershell)	`.\sds\Scripts\Activate.ps1`
Exit the virtual environment	`deactivate`

1.3.3. Install some packages.

```
pip install opencv-python
```

2. AWS Account

Ensure you have an active AWS account with sufficient credits.

Activity 10: People Counting Service using Edge-Cloud Architecture

In this activity, students will explore the performance differences among Edge, Cloud, and Edge–Cloud computing architectures through a people-counting use case. The state-of-the-art machine-learning model, **YOLOv8** (https://docs.ultralytics.com/models/yolov8), will be used to detect individuals in images uploaded from their computers.

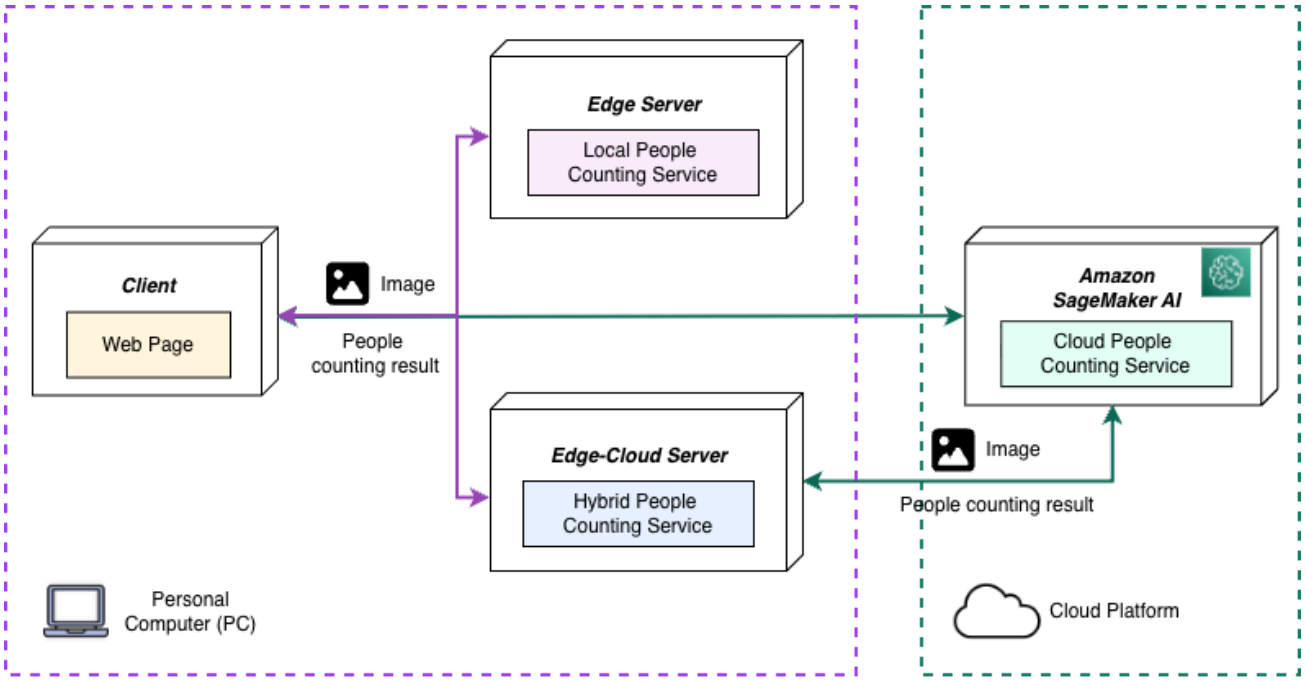


Figure 1: The system architecture of People Counting Service running on Edge, Edge-Cloud, and Cloud computing

As illustrated in Figure 1, the YOLOv8 models run locally on personal computers and on a cloud platform to represent Edge, Edge-Cloud, and Cloud computing configurations. The Edge configuration uses the lightweight YOLOv8n model to perform local people counting, whereas the Cloud configuration employs the YOLOv8x model, deployed as a cloud-based service on Amazon SageMaker AI.

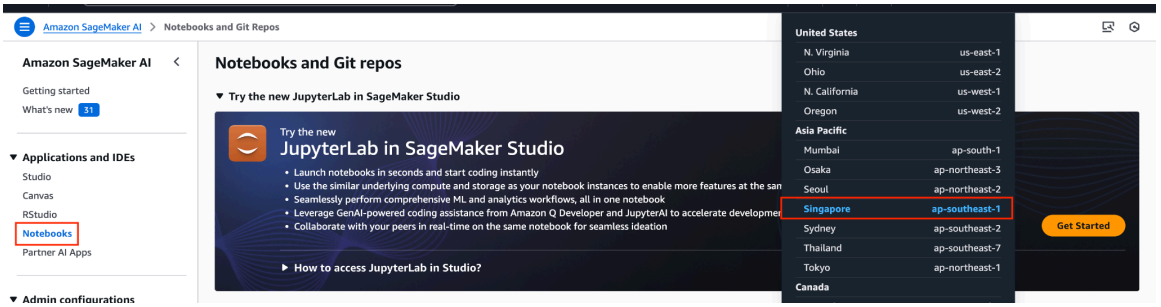
In the Edge–Cloud configuration, the hybrid people counting service uses the *confidence score* produced by the local model to determine whether a user request should be forwarded to the cloud service for further processing, potentially improving *detection accuracy*. The detection results are then displayed through a web application hosted locally on each student's computer.

Setup

1. Amazon SageMaker AI

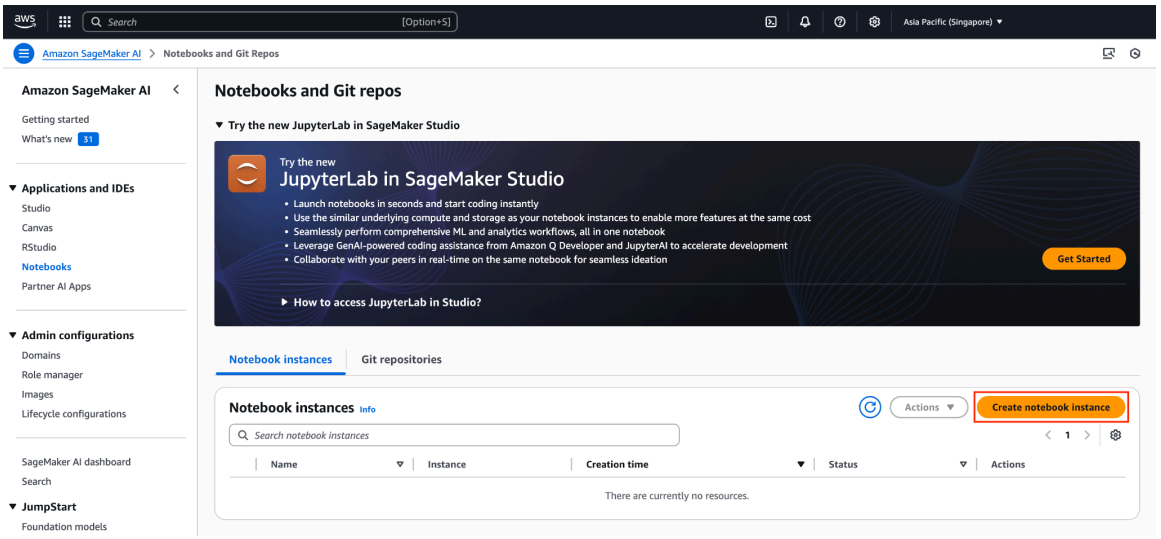
1.1. Open **Amazon SageMaker AI** (or Amazon SageMaker for some AWS accounts).

1.2. Under Applications and IDEs section, select **Notebook**. Ensure that your AWS region (`AWS-REGION`) is set to **Asia Pacific Singapore** (`ap-southeast-1`).

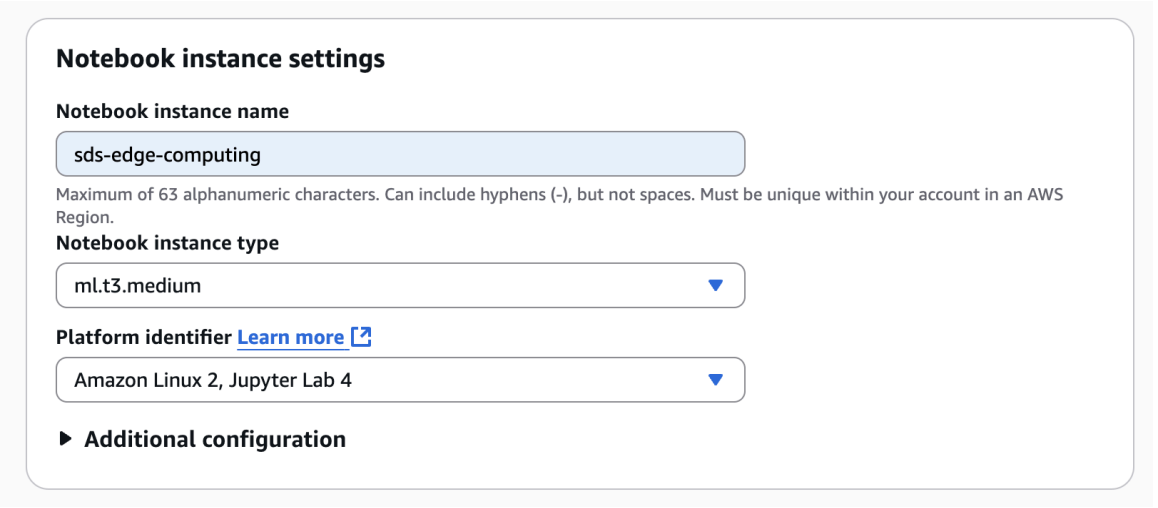


1.3. Create a SageMaker notebook instance.

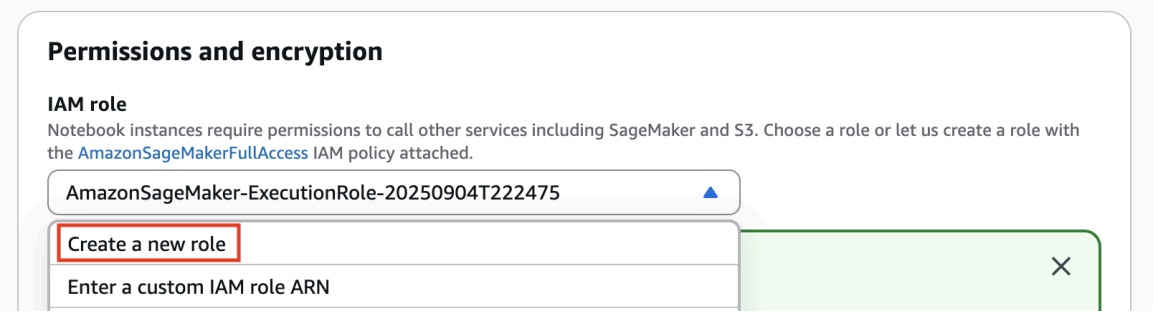
1.3.1. Click **Create notebook instance**.

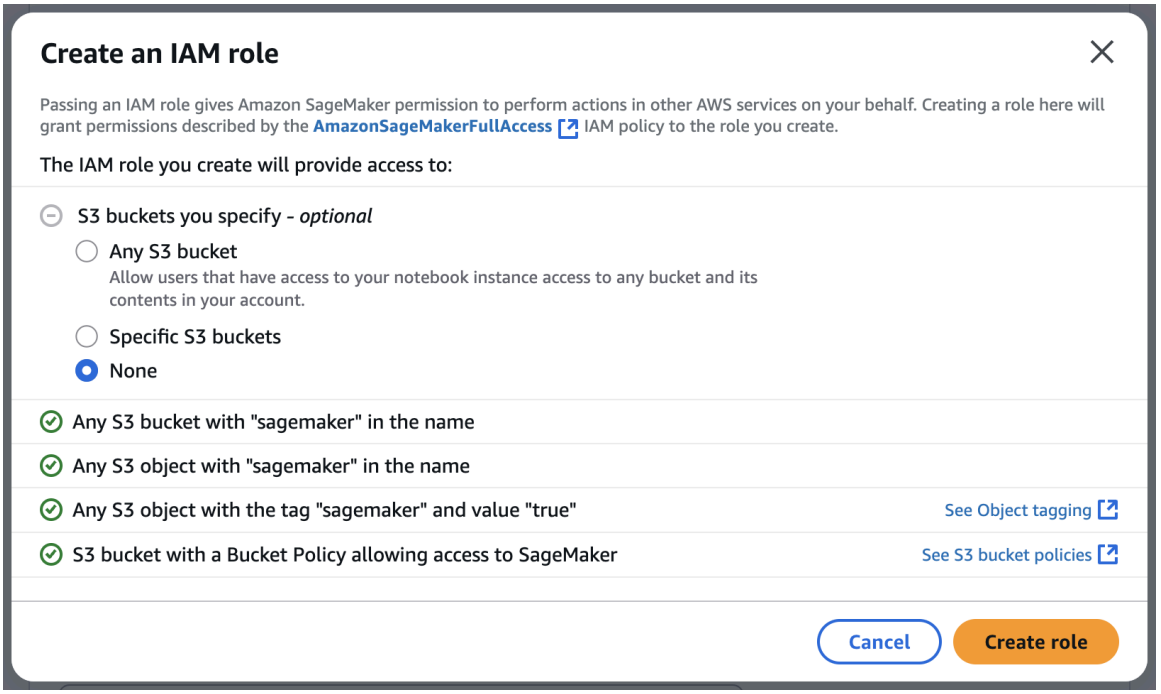


1.3.2. Configure the **Notebook instance settings** as shown below.

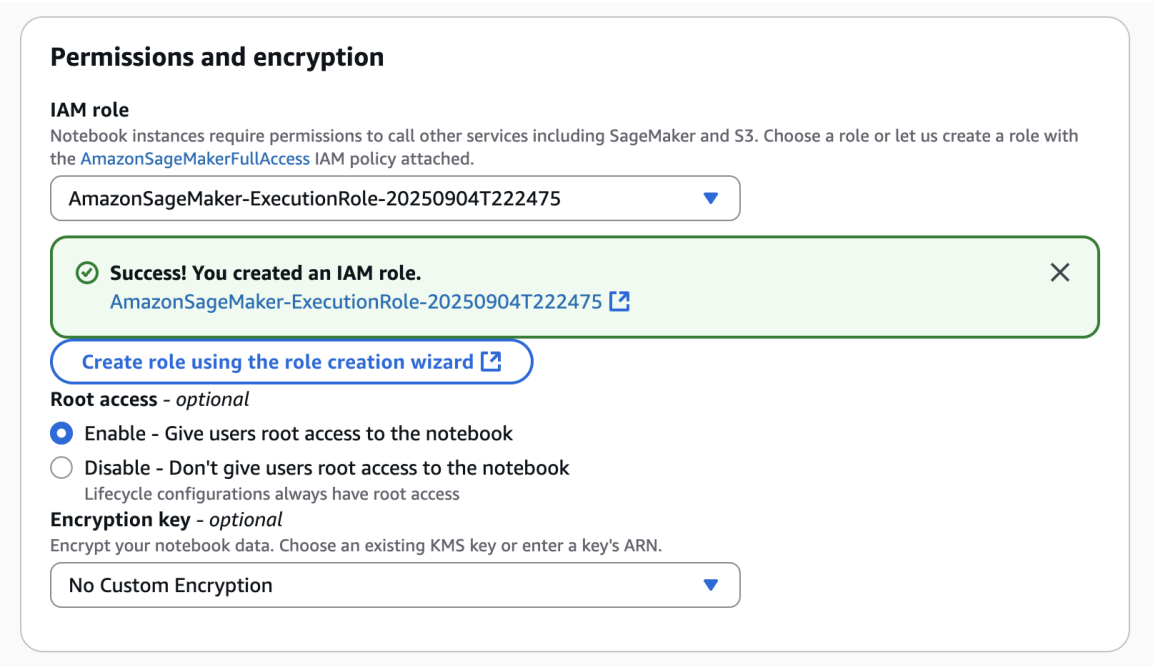


1.3.3. At the Permissions and encryption section, click **Create a new role**. Configure access as shown below, then click **Create role**.

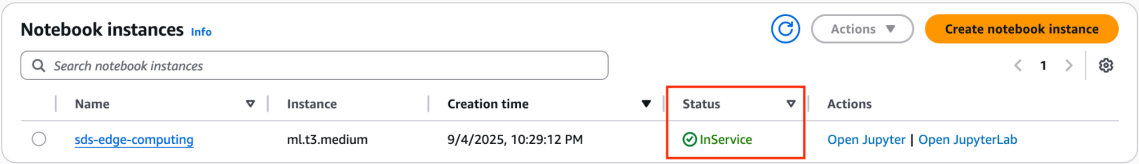




1.3.4. Ensure that the new IAM role is created then click **Create notebook instance**.

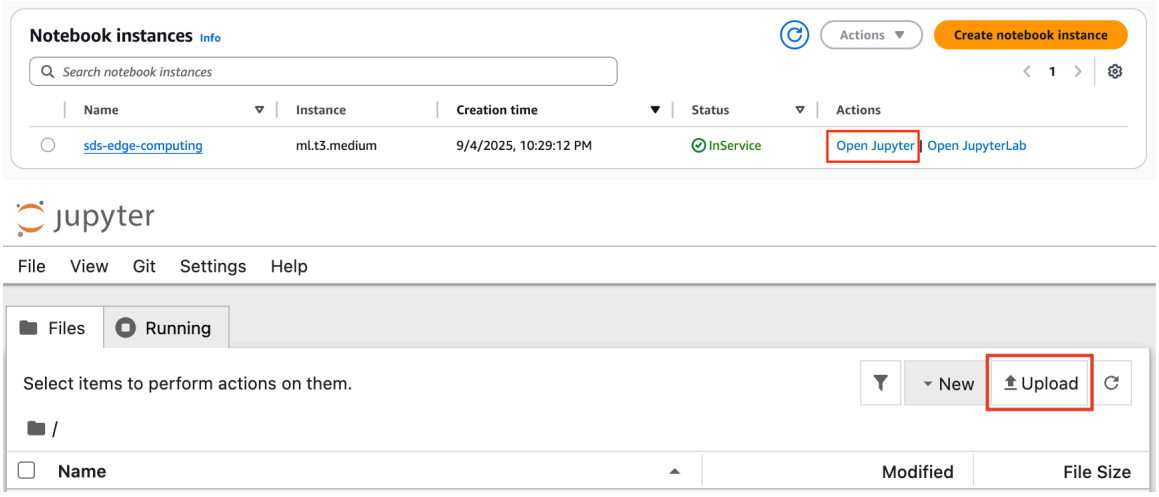


1.3.5. Wait until the notebook is created successfully, ensure the status is **InService**.

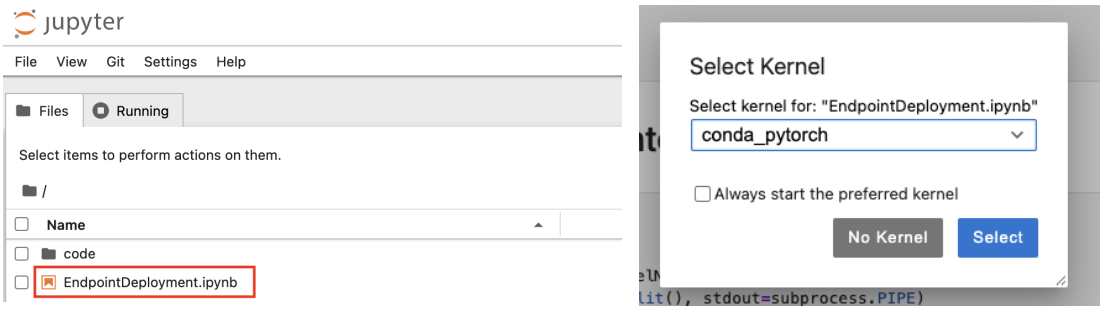


1.4. Deploy the model on SageMaker Endpoint.

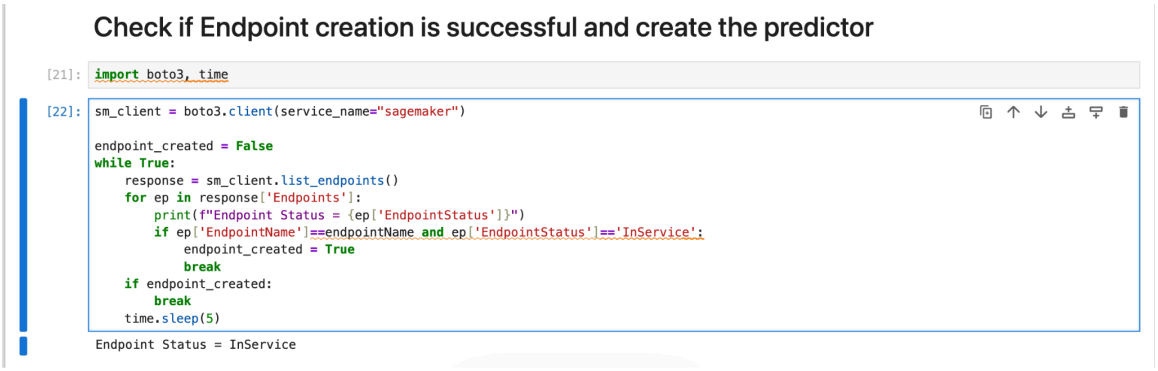
1.4.1. Open the created Jupyter Notebook and upload the files downloaded from the `sds-edge-computing` repository located in the `cloud` directory



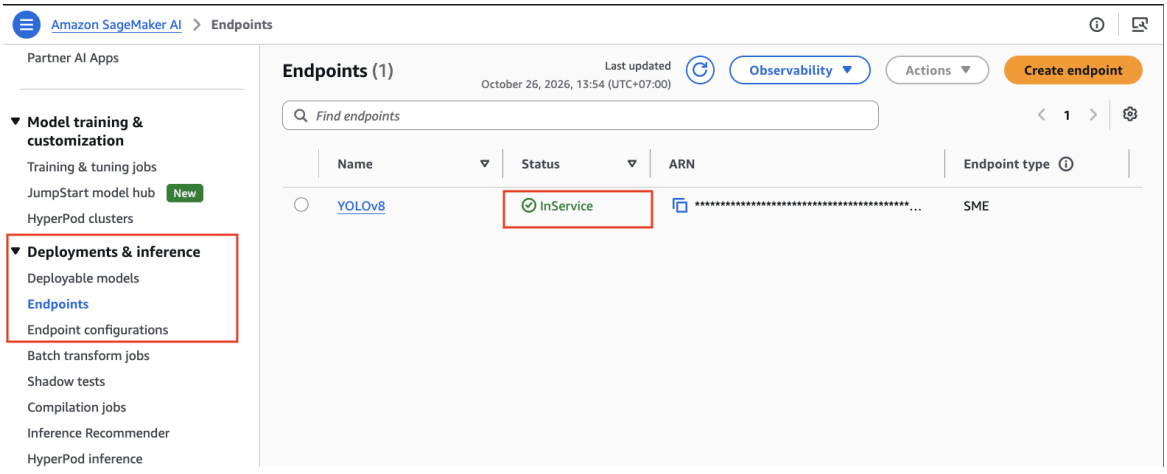
1.4.2. Open the **EndpointDeployment.ipynb** and *run all cells above the **Cleanup** part.*



1.4.3. Wait until terminal display output `Endpoint Status = Inservice`.



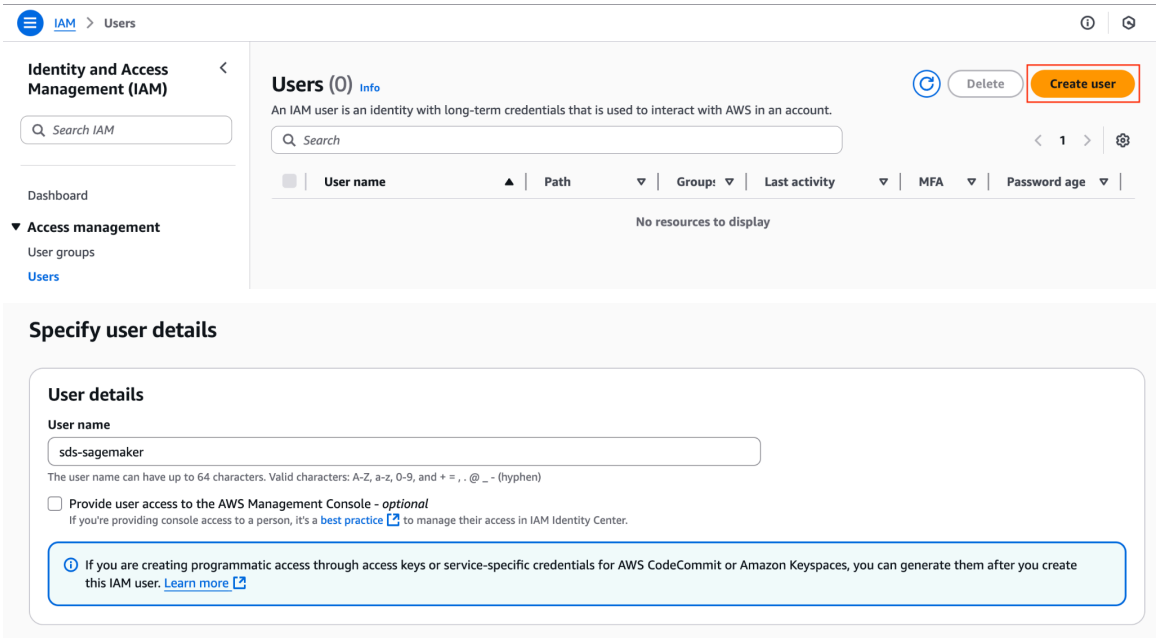
1.4.4. Verify the deployment status of *Deployable models*, *Endpoint*, and *Endpoint configurations* through the AWS console under Deployments & Inference.



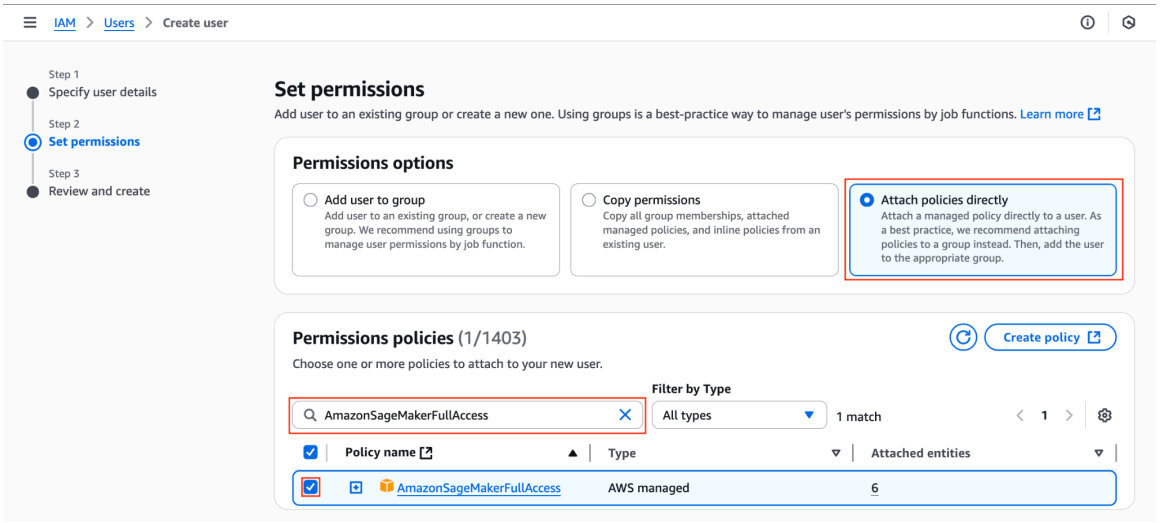
1.5. Create an IAM user for access to Models, Endpoint Configurations, and Endpoints in Amazon SageMaker AI.

1.5.1. Go to the IAM page.

1.5.2. Navigate to **Users** in the Access management section. Click **Create user** and specify user details.



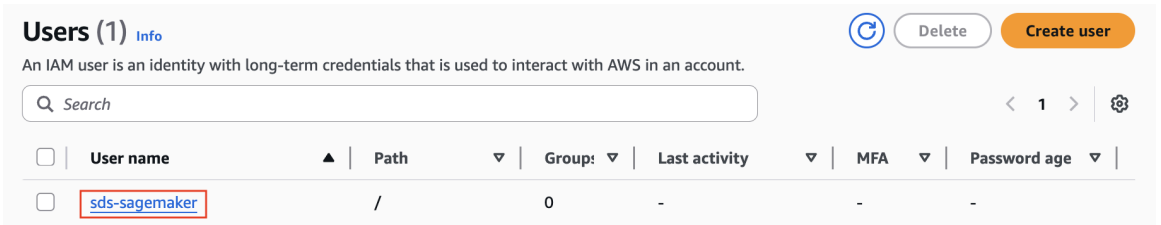
1.5.3. Attach the `AmazonSageMakerFullAccess` policy to the user. Review the configuration and click **Create user**.



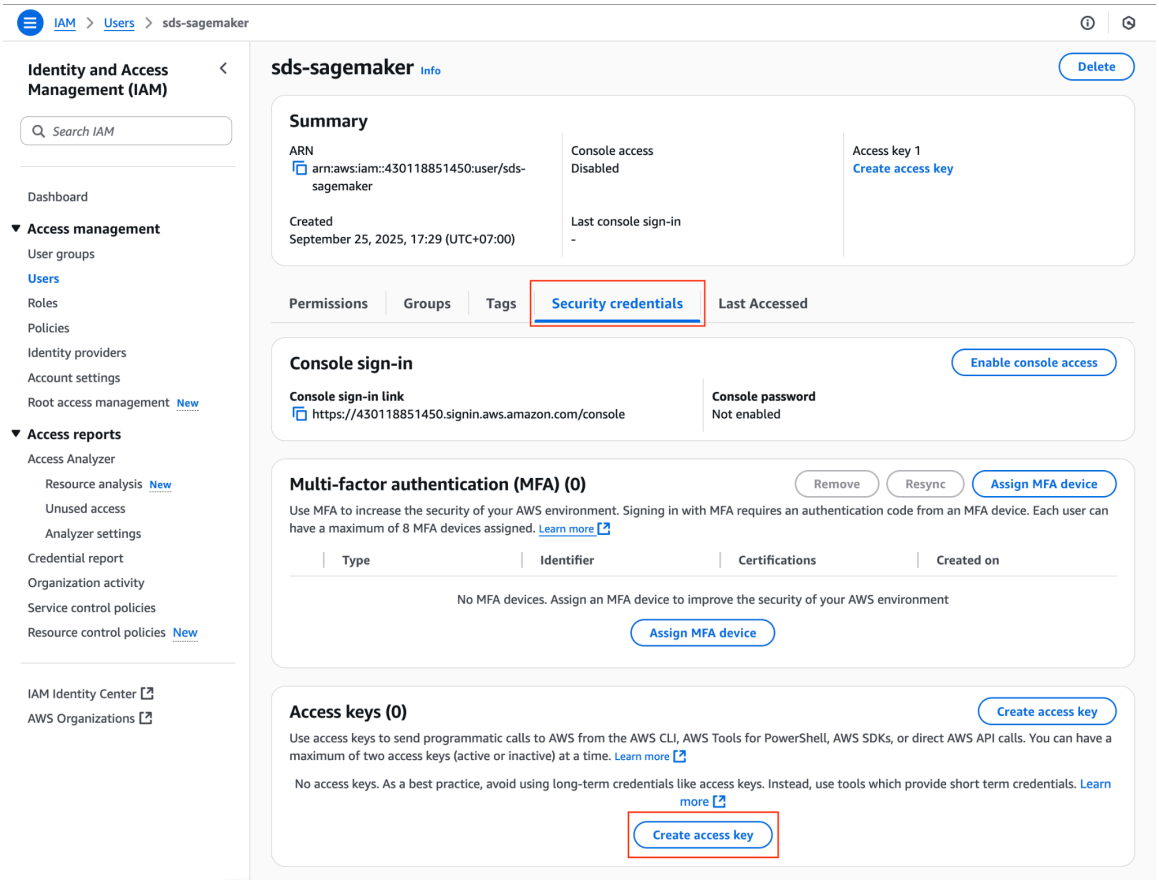
1.5.4. Wait until the user is created successfully.

1.6. Create an access key to authenticate your requests as the IAM user.

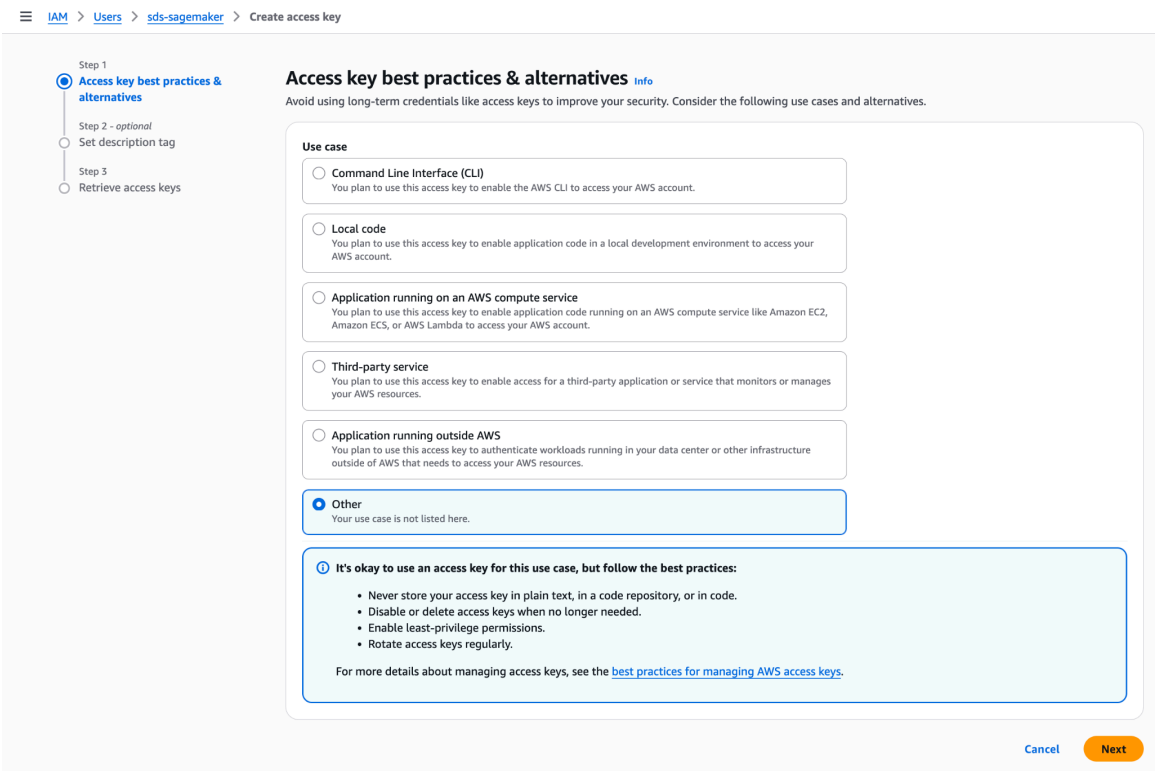
1.6.1. Go to the created user.



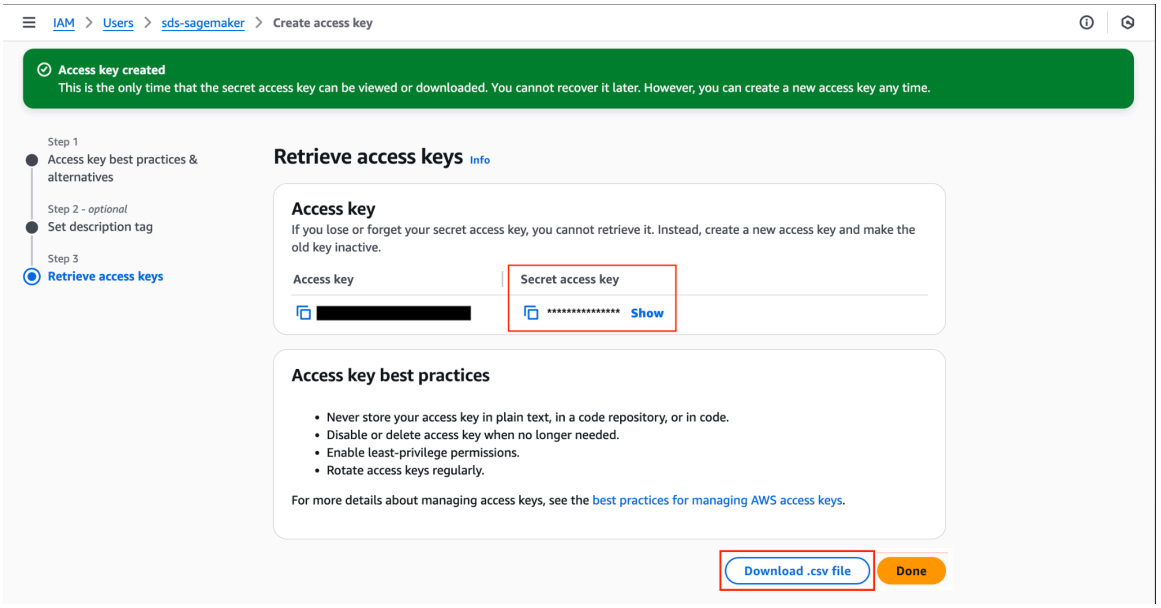
1.6.2. Go to **Secret credentials**. In the Access keys section, click **Create access key**.



1.6.3. Choose **Other** for this use case. Then click **Next**.



1.6.4. Save your access key (`ACCESS_KEY_ID`) and secret access key (`SECRET_ACCESS_KEY`) securely. *The secret access key cannot be retrieved if lost*. Once both keys are saved, click **Done**.



1.6.5. Wait until the key is created successfully, and check if its status is **Active**.

2. Personal Computer

2.1. In your PC, go to folder `sds-edge-computing`.

2.2. Create a `.env` file and fill in the parameter according to `.env.example`. Use the **`AWS_REGION`**, **`ACCESS_KEY_ID`**, and **`SECRET_ACCESS_KEY`** values obtained during the previous AWS setup steps.

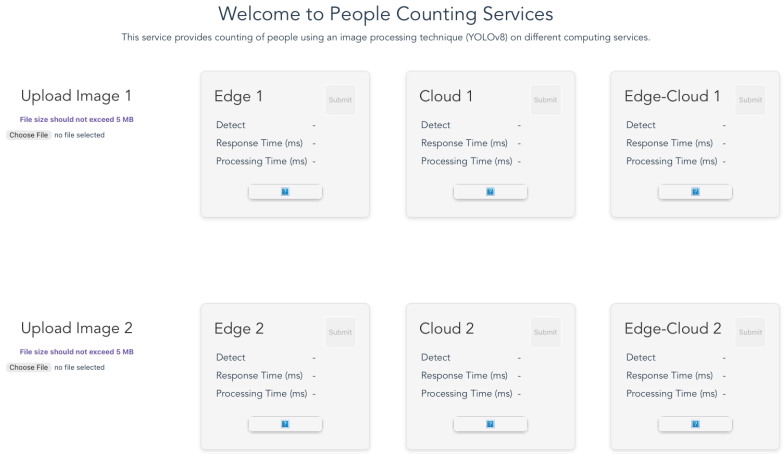
```
# Shared AWS credentials
AWS_ACCESS_KEY_ID=<ACCESS_KEY_ID>
AWS_SECRET_ACCESS_KEY=<SECRET_ACCESS_KEY>
AWS_REGION=<AWS_REGION>

# Client (Vite)
VITE_EDGE_URL=http://127.0.0.1:5001
VITE_HYBRID_URL=http://127.0.0.1:5002
VITE_CLOUD_REGION=${AWS_REGION}
VITE_ACCESS_KEY_ID=${AWS_ACCESS_KEY_ID}
VITE_SECRET_ACCESS_KEY=${AWS_SECRET_ACCESS_KEY}
VITE_ENDPOINT_NAME=YOLOv8
```

2.3. Open Docker Desktop, then run the following command to start all three services. This process may take some time.

```
docker compose up --build
```

2.4. After the Docker build completes, open http://localhost:5173/ in your browser. You may find the "Welcome to People Counting Services" webpage.



Assignment

1. Run the people counting services

1.1. Select **`Choose File`** and upload photos using the web client
- For image no. 1-4, using the given photos (1-4.jpg) inside the activity folder.
- For image no. 5-10, capture people in the classroom with a different number of people and also record the actual number of people to use as ground truth later.

1.2. Select **`Submit`** to process each image using all three people counting services.

2. Record the Results

2.1. Open **`People counting record sheet.xlsx`** in the **`sds-edge-computing`** folder.

2.2. Record the detection results for each request, including: Result image, Number of people detected, Processing time (3 trials), and Response time (3 trials).
You may ignore the first few requests if they produce outliers during initialization.

2.3. Capture screenshots of the result images and insert them directly into the cells using **Insert → Image → Insert Image in Cell** to keep the file under mCV upload limit of 20 MB.

2.4. Complete all yellow-highlighted fields in the spreadsheet.

Submission

mCV: Attach files

1. Screenshot of AWS SageMaker AI (Jupyter Notebook).
2. Screenshot of the "Welcome to People Counting Services" webpage.
3. Completed **`People counting record sheet.xlsx`**.

mCV: Answer the questions

1. **Performance comparison:** In the result summary section (at the end of the record sheet), observe the values of **Average Response Time (ms), Average Processing Time (ms), and Average Accuracy**. Identify which set up (Edge, Cloud, or Edge–Cloud) shows highest or lowest values for each metric. Analyze the reasons behind these differences.
2. **Suitable use cases:** Based on the results, which characteristics, needs, or requirements are **best suited** for each setup (Edge/Cloud/Edge–Cloud), considering the accuracy, response time, and cost trade-offs?
3. **AWS cleanup:** Have you completed the **cleanup step** after finishing the assignment to avoid unexpected AWS charges?

Cleanup

After completing the activity answer sheet, save all required results in **`People counting record sheet.xlsx`** for submission. Once you have saved your work, **cleanup** all AWS resources created during this activity to avoid unexpected charges.

1. Personal Computer

1.1. In your PC, go to folder **sds-edge-computing** and use the following command to remove and destroy the running containers:

```
docker compose down
```

2. Amazon SageMaker AI

2.1. Run the code in the **Cleanup** part. This step may consume time.

Cleanup by removing Endpoint, Endpoint Config and Model

```
In [ ]: response = sm_client.describe_endpoint_config(EndpointConfigName=endpointName)
print(response)
endpoint_config_name = response['EndpointConfigName']

# Delete Endpoint
sm_client.delete_endpoint(EndpointName=endpointName)

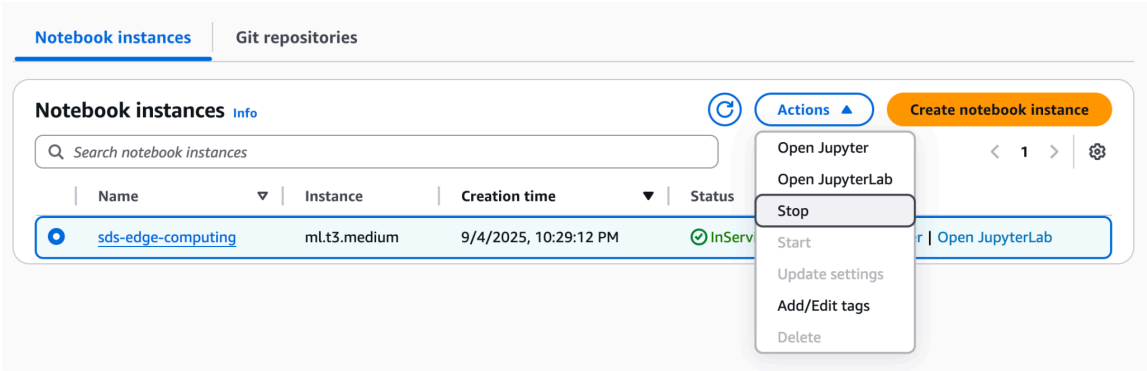
# Delete Endpoint Configuration
sm_client.delete_endpoint_config(EndpointConfigName=endpoint_config_name)

# Delete Model
for prod_var in response['ProductionVariants']:
    model_name = prod_var['ModelName']
    sm_client.delete_model(ModelName=model_name)
```

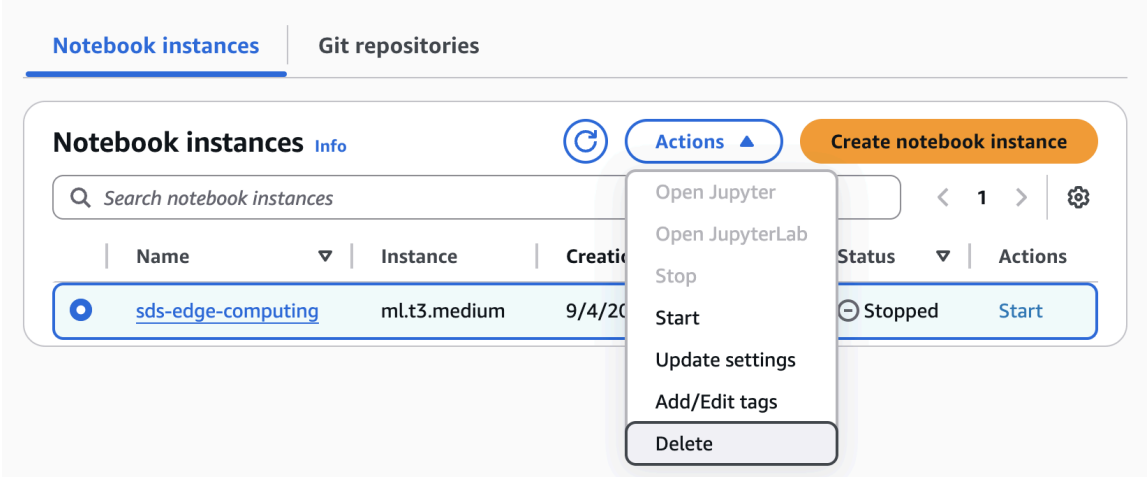
2.2. The deployed *Deployable Model*, *Endpoint configurations*, and *Endpoints* under Deployments & Inference should be removed.

2.3. Follow https://docs.aws.amazon.com/sagemaker/latest/dg/ex1-cleanup.html

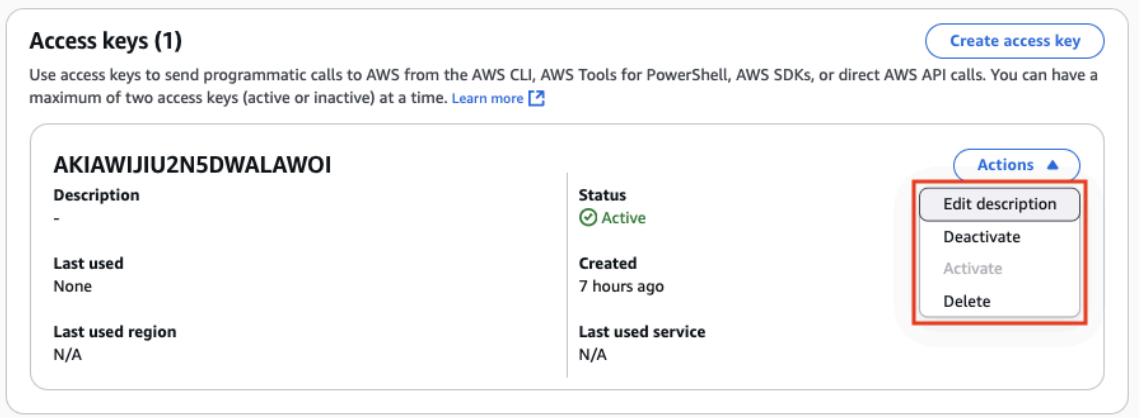
2.4. Stop the Jupyter Notebook. Wait until its status is **Stopped**.



2.5. If the notebook is no longer in use, delete it to avoid any potential future costs.



2.6. Deactivate or delete your access key in the IAM/Access management/Users.



2.7. Delete IAM user.

